

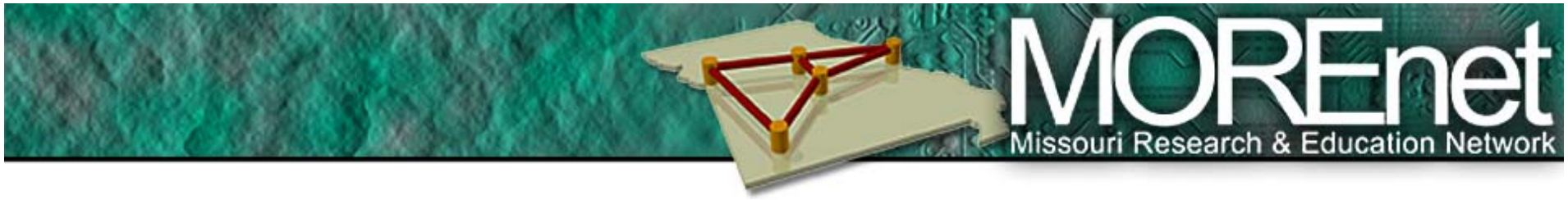
Parallel Development and Hosting Using Apache, Tomcat, and MySQL

Garrel Renick
garrel@more.net

http://www.more.net

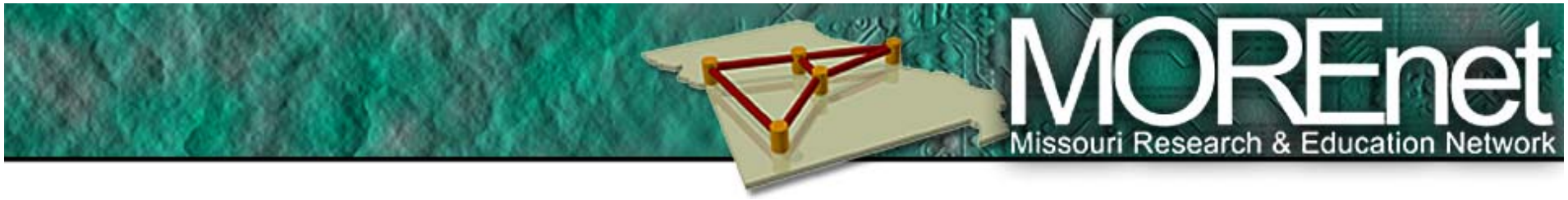
University of Missouri System

Copyright ©2003 MOREnet and The Curators of the University of Missouri



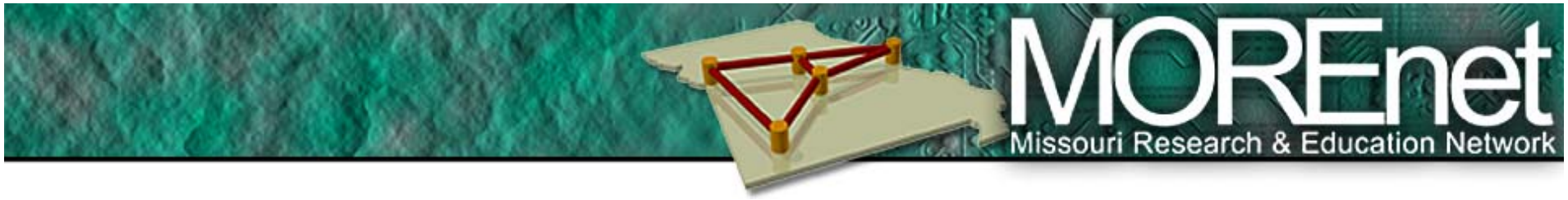
Overview

In this session we'll examine parallel desktop/server environments for web site and application server hosting. Both environments are built from Apache, mod_jk, Tomcat, and MySQL, but are OS-independent. Configuration choices were guided by security concerns, developer usability, and the need for applications to function without modification in both environments.



What We'll Cover

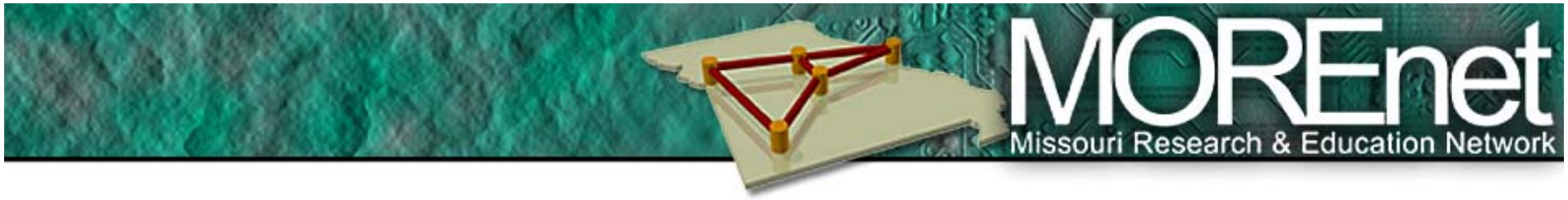
- Creating a hosting environment is a big topic—too big for a one-hour session.
- We'll focus on choices that affect the customer's ability to develop content on the desktop that functions like it will on the production servers.



What We'll Cover

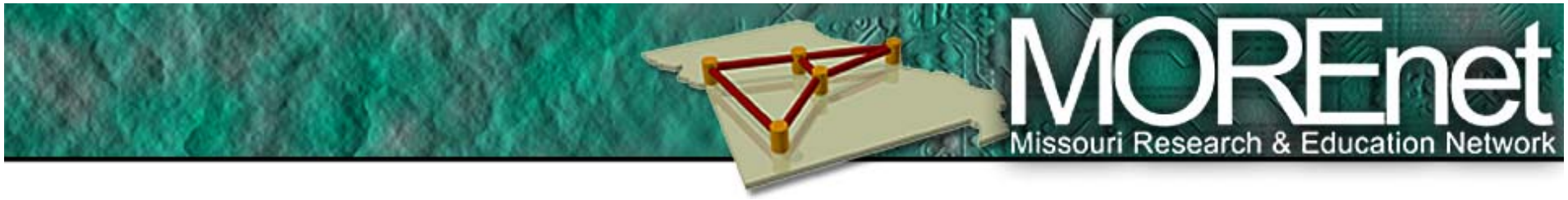
We'll examine five areas:

- Why Apache, Tomcat, MySQL and mod_jk?
- Configuring a Usable Site Structure
- Integrating Web Sites and Web Applications
- Providing Database and Mail Services
- Managing Tomcat
- Strategies for Helping Customers



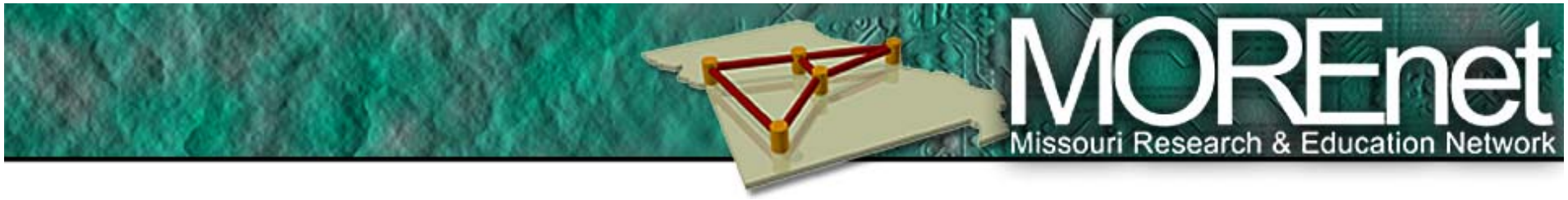
Why Apache, Tomcat, MySQL and mod_jk?

What problem are we trying to solve?



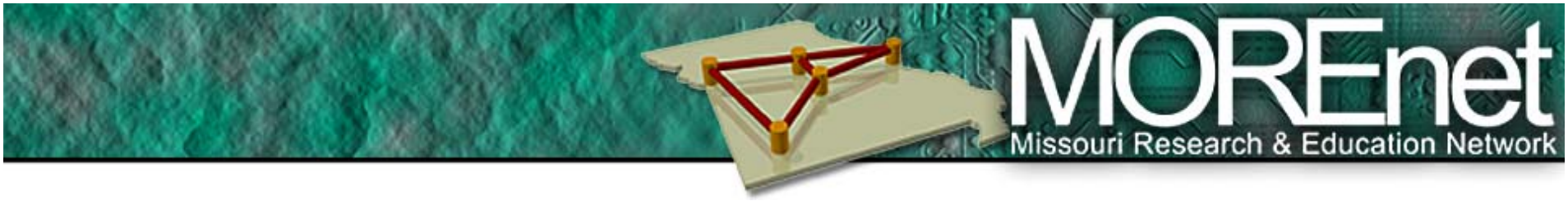
Typical ISP Web Hosting Services

- Web site domain hosting (static content)
- Web application hosting (dynamic content)
- Database services
- Security services



Other Hosting Services

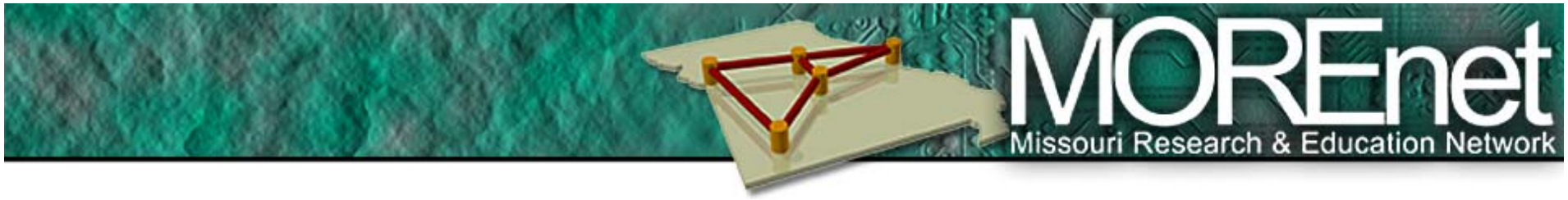
- Hosting isn't just for ISPs.
- Many organizations provide internal hosting services. Application developers and web site designers must develop in similar ways as ISP customers—the environment is defined and fairly unchangeable.
- Contracted developers are also bound to environment choices made by the host administrators.



Apache, Tomcat, MySQL and mod_jk

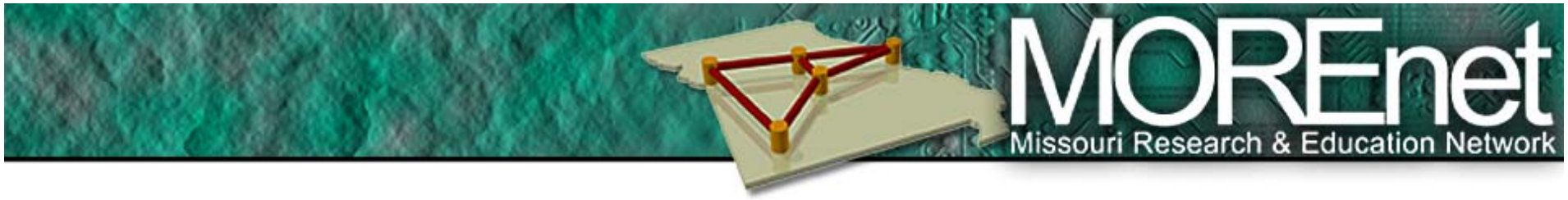
Fulfill these roles:

- HTTP server
- System Application Server
- Customer Application Server
- System Database
- Customer Database
- Web Site/Application Server Connector



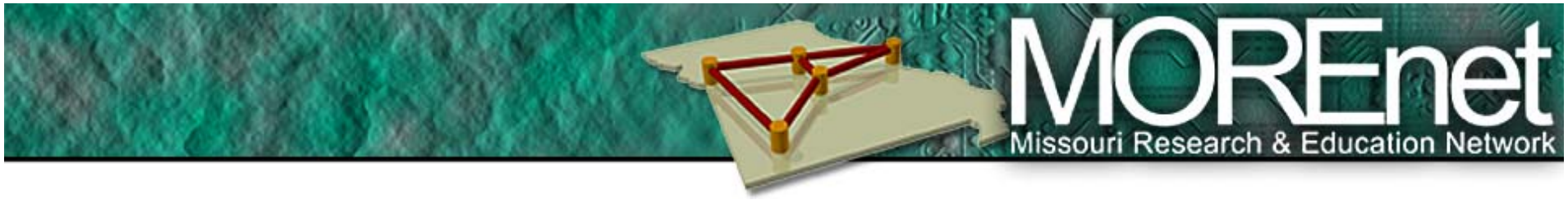
Apache, Tomcat, MySQL and mod_jk

- When a system is configured, administrators must balance hosting concerns with concerns of the client or customer.
- Decisions affect security, usability, management, support, etc.



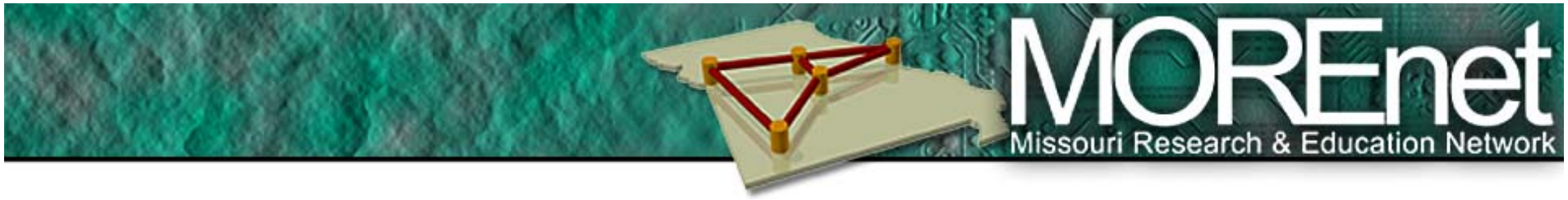
Hosting Concerns

- Systems must be distributed for scaling.
- Web sites and web applications must be virtually hosted due to limited resources and to decrease management costs.
- Administrative support must be delegated to customers as much as possible to decrease costs and support time.
- Customers frequently do not need all available services.



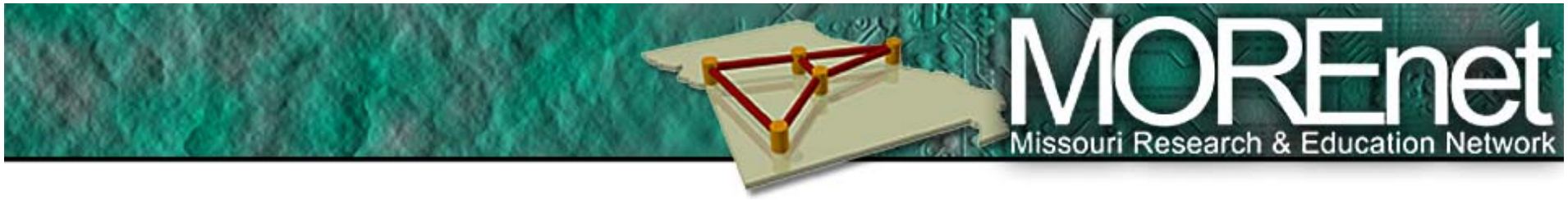
Hosting Concerns

- Individual customer web sites and web applications must be isolated from each other.
- Customers cannot be given shell access but they must manage content.
- Customers cannot be trusted to build security features into their web applications.
- Web Applications must be prevented from harming system resources.



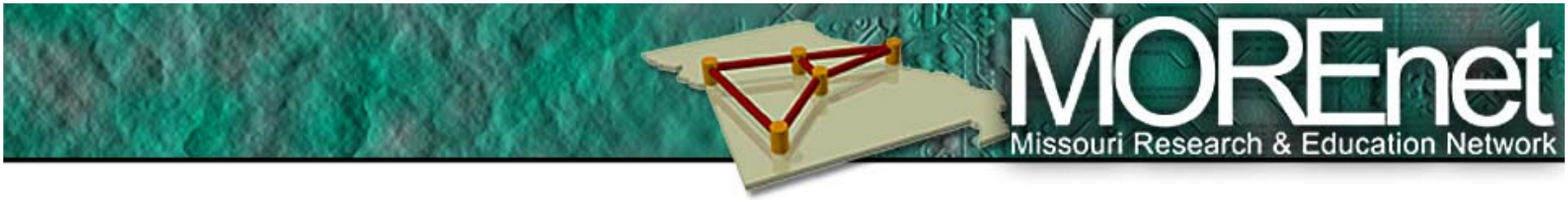
Customer Concerns

- Access to the site using common tools like FTP to update web site content.
- Access to web site and web application logs.
- Access to manage their databases.



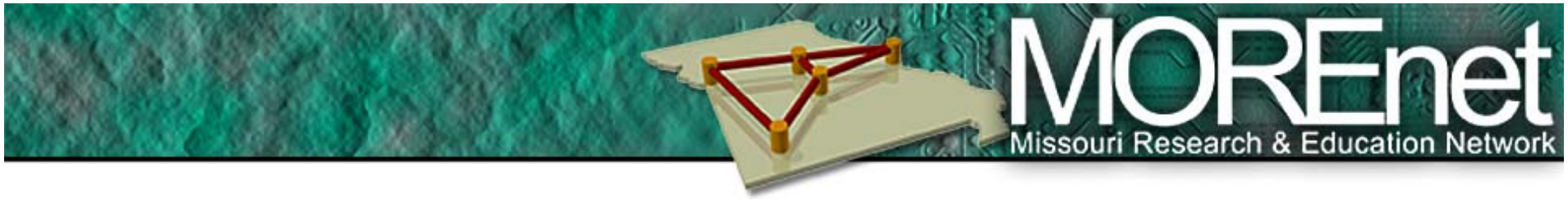
Customer Concerns

- Content Management
- Versioning
- Design issues—web site designers may need the ability to integrate web site and web application look and feel to simplify development and create consistency. Common headers, footers, navigation, etc.



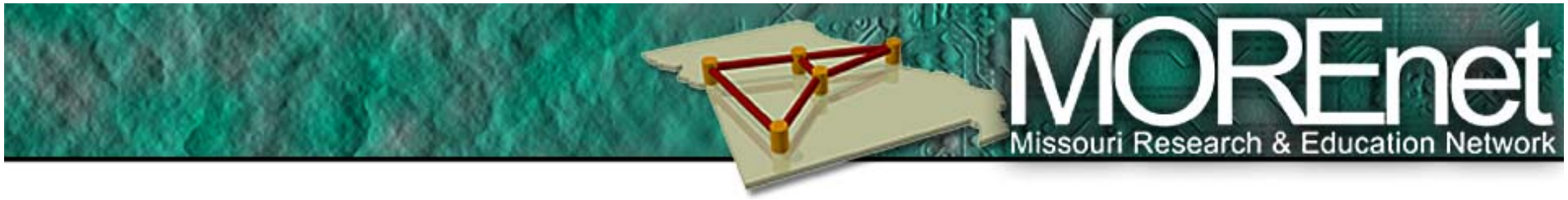
Customer Concerns

- Prefer single sign-on.
- Some customers have multiple web sites and prefer the ability to manage them using the same accounts
- Have very limited funds for development tools and test systems.
- Does the host system provide well-documented features?
- Are the technologies supported by tool vendors?



Do Apache, Tomcat, MySQL & mod_jk Provide a Solution?

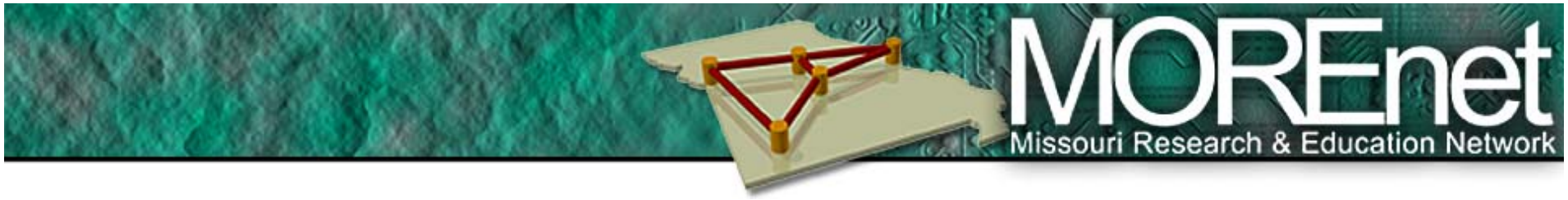
- All are open-source technologies that are free or inexpensive, and supported by developer communities or low-cost support contracts.
- A successful implementation requires that the products resolve host and customer concerns.



Apache 2

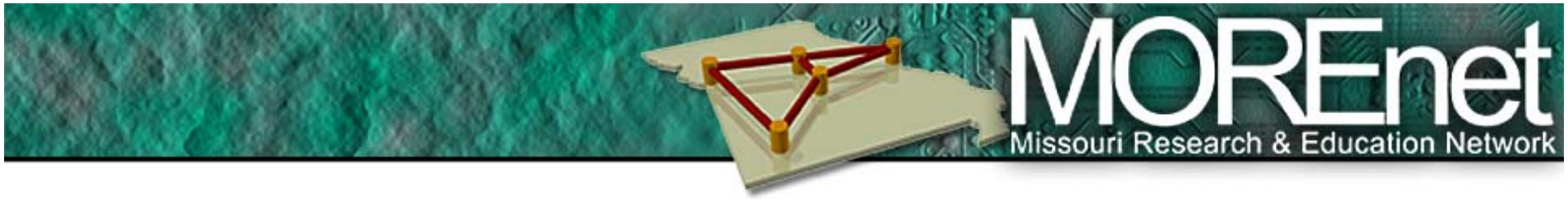
Resolves the following concerns:

- Supports virtual hosting and easily configured to isolate content
- Scalable using proven technologies
- Proven track record for performance and can be configured to serve static content in web applications



Apache 2

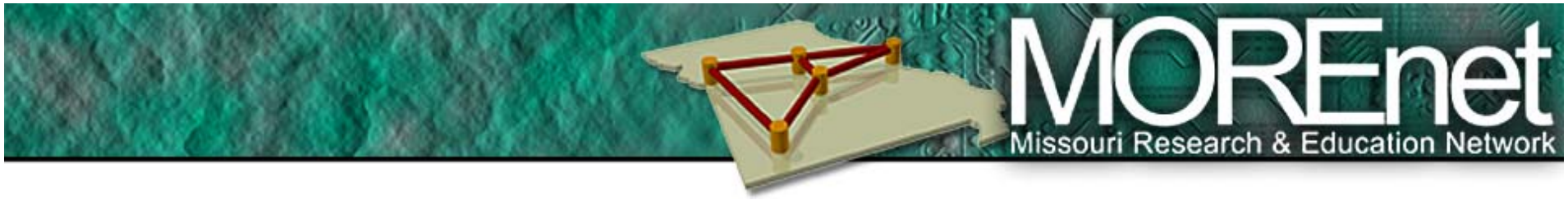
- Very configurable and easily integrated with an application server
- Runs on a variety of platforms
- Supports web standards



Tomcat 4.1

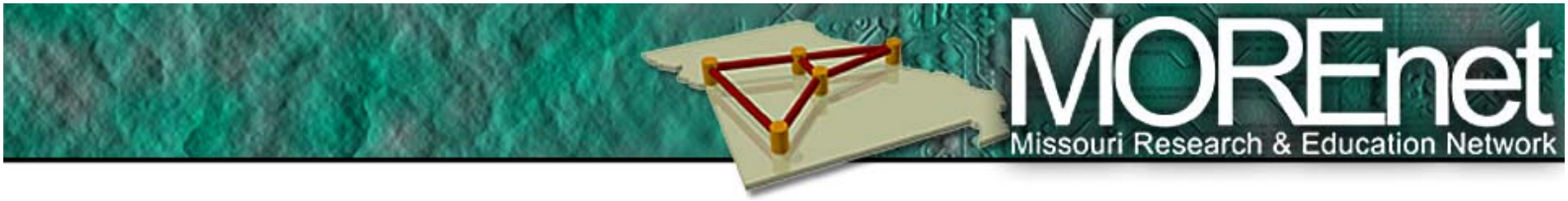
Resolves the following concerns:

- Supports virtual hosting and is scalable
- Very configurable and easily integrated with a web server
- Supports the Servlet 2.3/JSP 1.2 specifications supported by commercial tool vendors



Tomcat 4.1

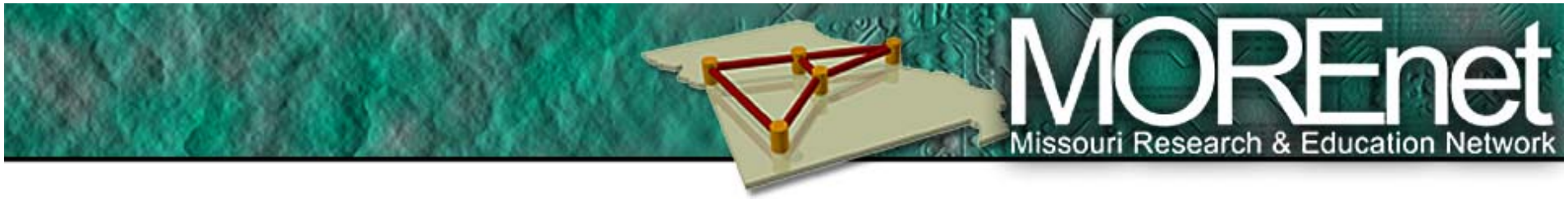
- Contains a manager application for administering webapps
- Supports the Java SecurityManager for detailed security of server
- Runs on a variety of platforms



MySQL

Resolves the following concerns:

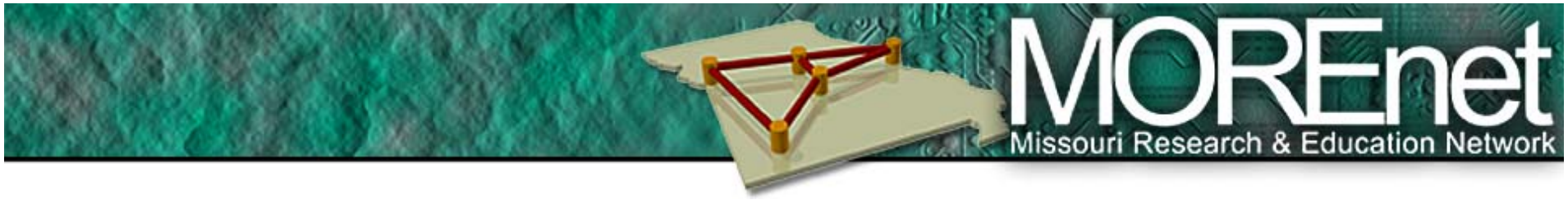
- Can be administered remotely without shell access.
- Proven track record for performance
- Runs on a variety of platforms
- Vendor support is provided with a nominal fee.



mod_jk

Resolves the following concerns:

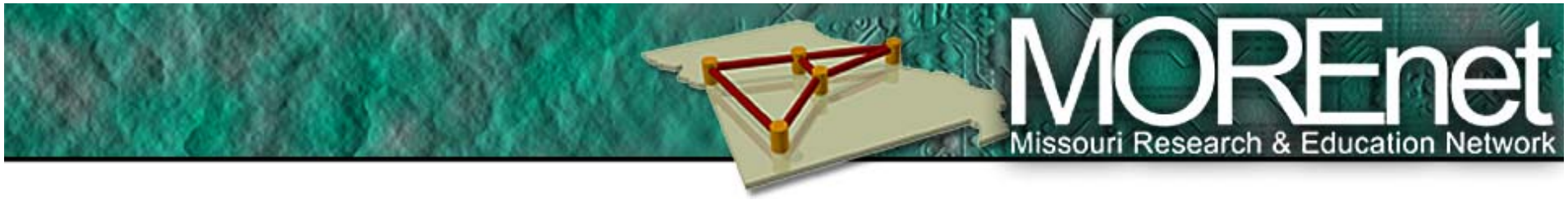
- Provides connection between Apache and Tomcat for seamless integration of web server and application server
- Runs on a variety of platforms



Do Apache, Tomcat, MySQL & mod_jk Provide a Solution?

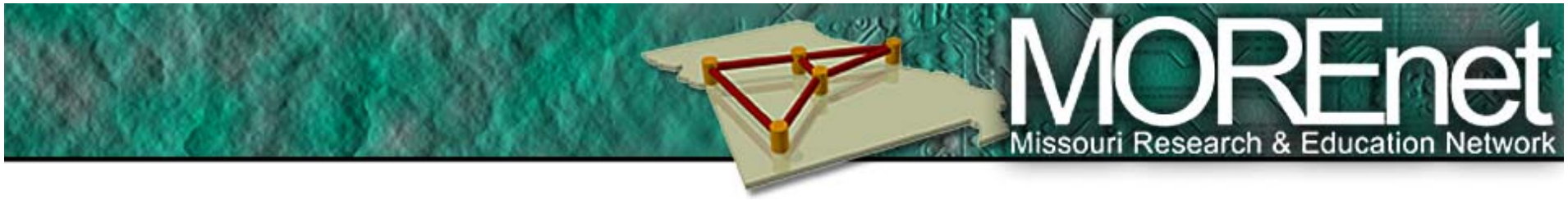
MOREnet uses them to support:

- Over 340 domains
- Almost 60,000 accounts
- Hundreds of account administrators
- Over 1500 web publishers
- Over 15 million page views per month



Making it Work

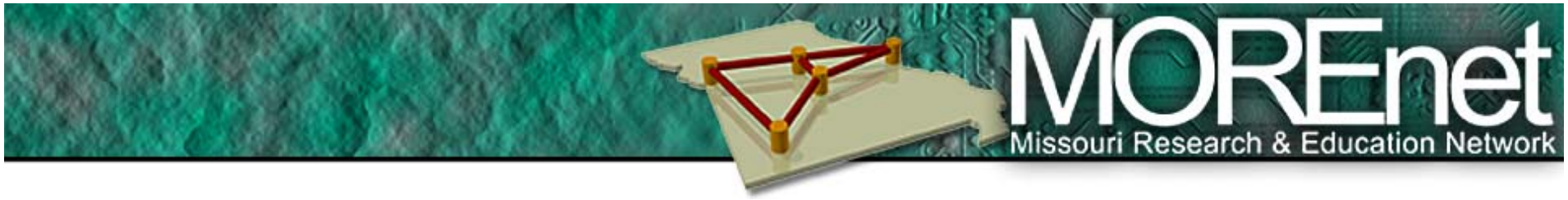
Using this background, let's examine configuration choices that affect the customer's ability to develop on the desktop and deploy on the production servers without modifications.



Configuring a Usable Site Structure

The possibilities are endless, but remember these goals:

- The structure must support virtual hosting and isolate customer content
- Web sites, web applications, and logs must be accessible via common tools
- Site access should be delegated to the customer
- Customers may have multiple web sites



Host—Site Structure

MOREnet chose the following structure:

`/web/k12/odessa/www.odessa.k12.mo.us`

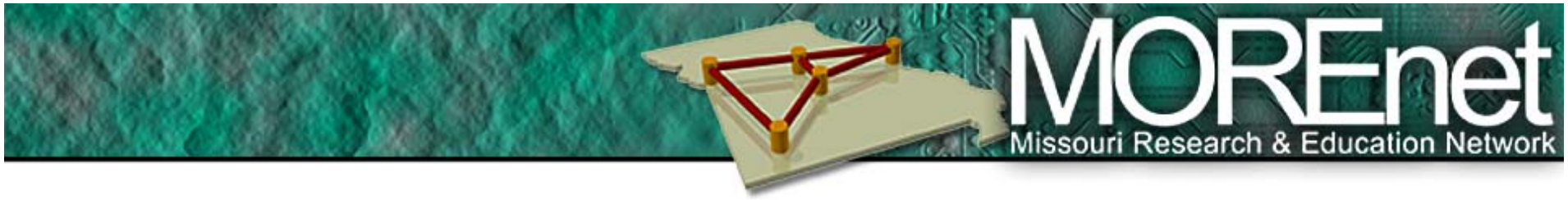
… `/www` (web site document root)

… `/logs` (web server logs)

… `/tomcat`

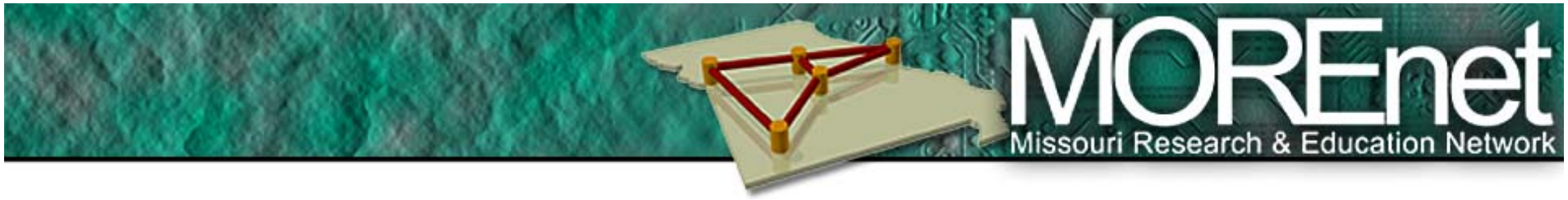
… `/tomcat/logs` (application server logs)

… `/tomcat/work` (application server work files)



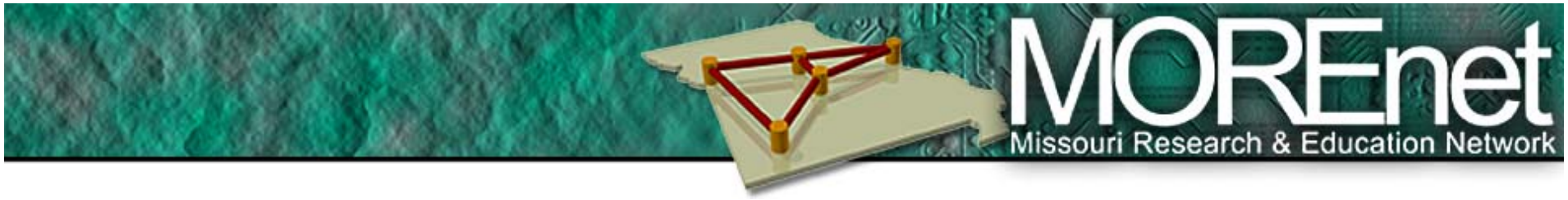
Host—FTP Access

- Customers use FTP to publish documents to document root. Anonymous FTP is not supported. Authentication by e-mail address.
- Access authentication by ProFTPD with `.ftpaccess` files.
- Access rules (i.e. publishing permissions) are automated by a custom administration tool used by customers.
- Files may exist on various systems, but the symbolic links resolve and appear in the FTP client under a `WebPublish` directory.



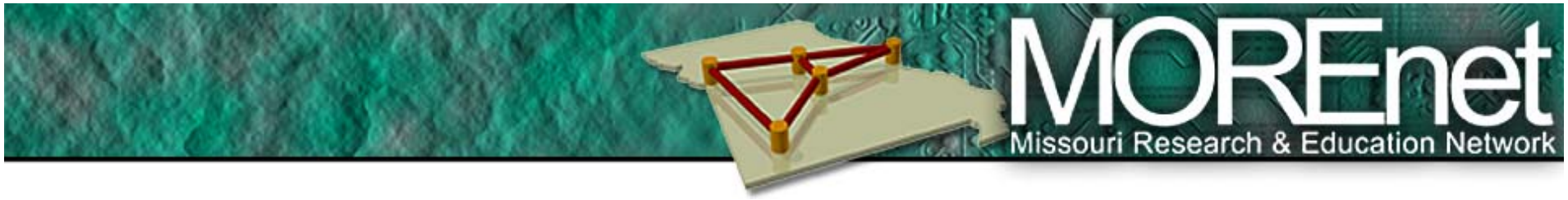
Host—File Permissions

- Read/write permissions only provided in the web site document root and below.
- Read-only permissions above web site document root for customer access to web server and application server logs.
- Note that Tomcat is in the same group as customer accounts so it can write files within the document root…this will make sense later in the presentation.



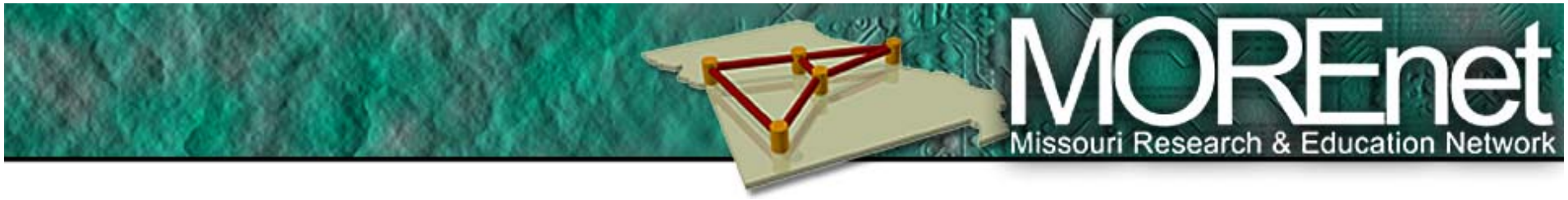
Host—Apache Logs

Each virtual host is configured to store log files in a **`logs`** directory at the same level as the **`www`** directory (web site document root). File permissions are set to read only for access via FTP.



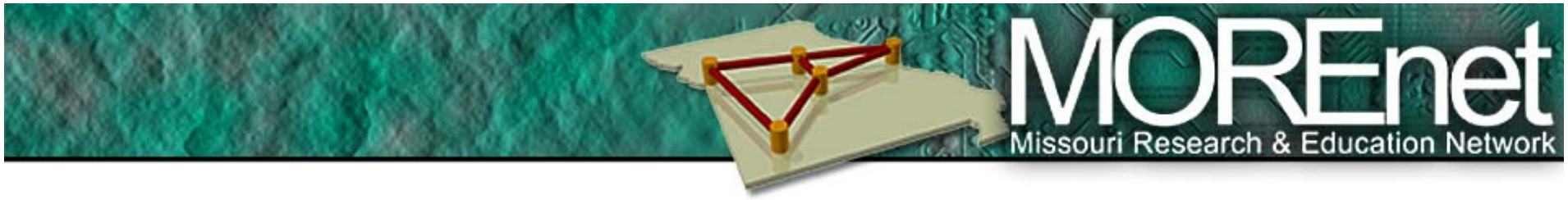
Host—Tomcat Logs

Each virtual host is configured to store logs files in a **`tomcat/logs`** directory at the same level as the **`www`** directory. File permissions are set to read only for access via FTP.



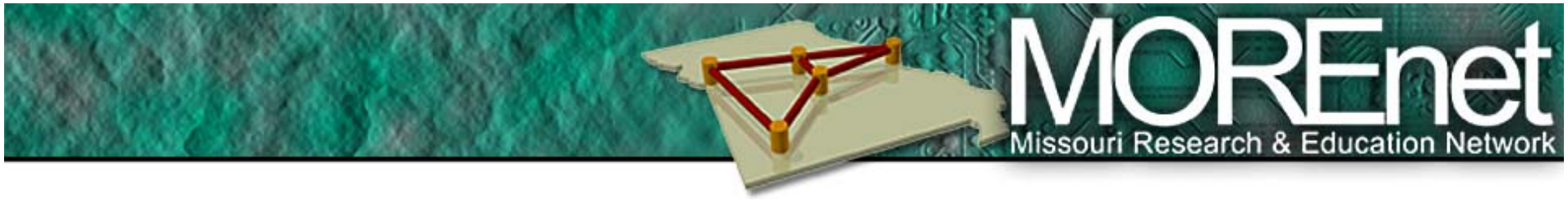
Host—Tomcat Work Files

Each virtual host is configured to store its work files in a **`tomcat/work`** directory at the same level as the **`www`** directory. File permissions are set to read only for the customer.



Desktop—Site Structure

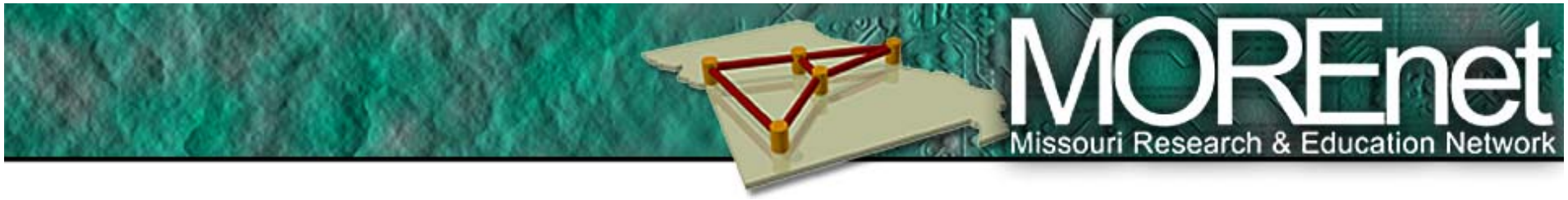
- Default distributions for Apache and Tomcat have web site files and logs scattered in the installations.
- To prevent confusion, the desktop site structure should be analogous to what the customer uses on the production systems.
- Configuration files can duplicate the host site structure for the localhost.



Desktop—Site Structure

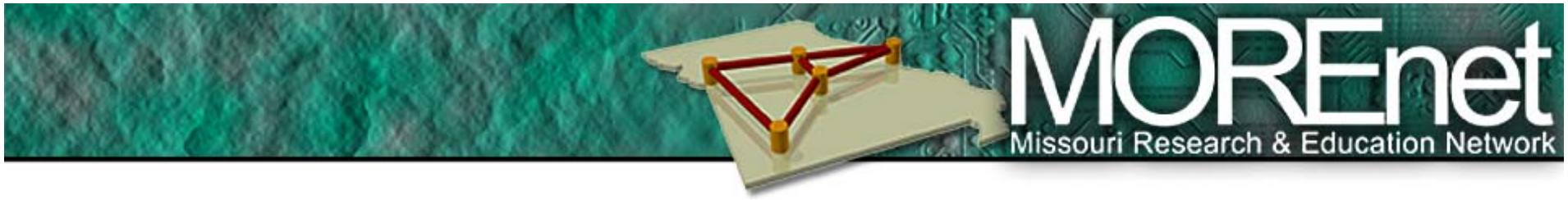
For example:

```
/WebPublish/localhost/
/WebPublish/localhost/www
/WebPublish/localhost/logs
/WebPublish/localhost/tomcat
/WebPublish/localhost/tomcat/logs
/WebPublish/localhost/tomcat/work
```



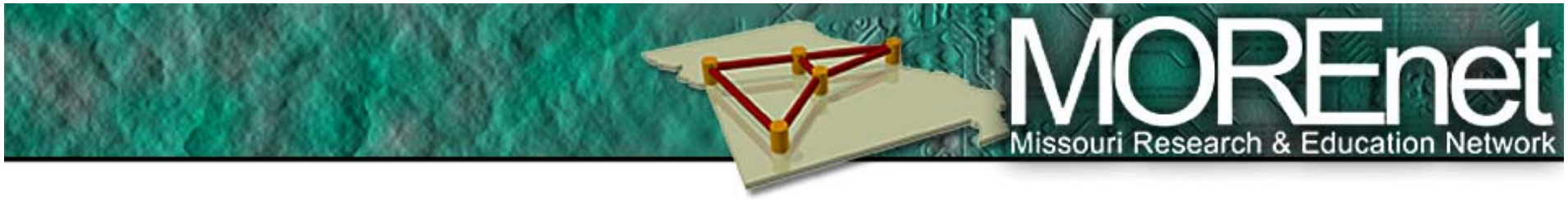
Integrating Web Sites and Web Applications

- Web sites and web applications should appear seamless to site visitors.
- Site designers frequently use Server Side Includes to use common components across a site.
- Customers should be able to modify application source files on the server if needed just like web site files, but source files should not be exposed by the site.



Integrating Web Sites and Web Applications

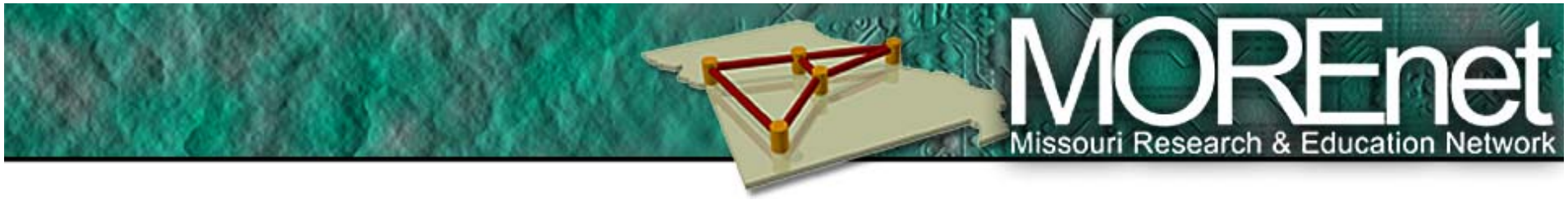
- Deployment of new web applications should be as hassle-free as possible.
- Apache should serve static content and only pass requests to Tomcat for dynamic content.



Integrating Web Sites and Web Applications

How do you do this?

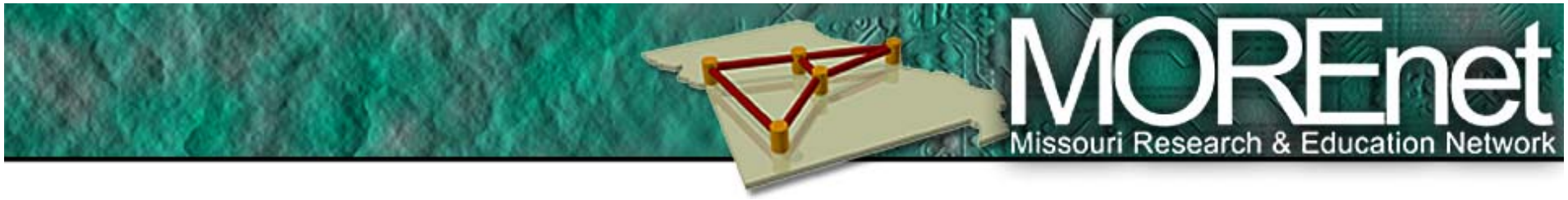
- Share the web site docBase and webapp appBase
- Configure Apache to process SSI in web application files
- Leverage mod_jk to make the integration seamless



Sharing the web site docBase and webapp appBase

By sharing the web site docBase and webapp appBase, customers benefit from:

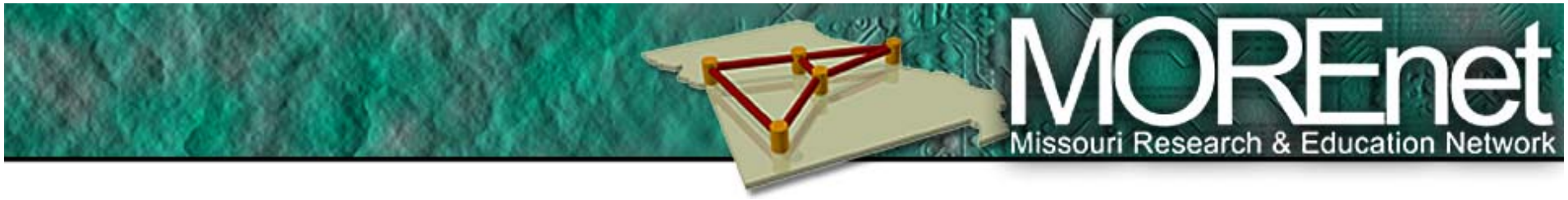
- Web site content and applications peacefully coexist.
- Web site content and web applications can share common components such as images, CSS, HTML fragments, etc.
- Site developers can use FTP to modify web site and web application content.



Sharing the web site docBase and webapp appBase

Apache sample configuration:

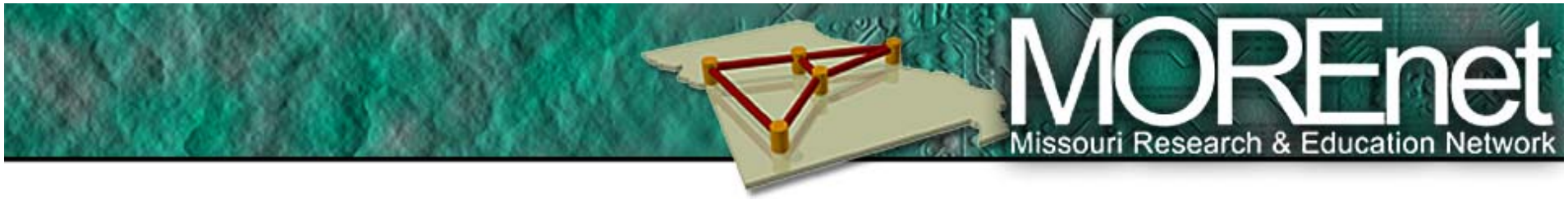
```
<VirtualHost localhost>
   ServerAdmin userid@domain
   DocumentRoot "/WebPublish/localhost/www"
   ServerName localhost
```



Sharing the web site docBase and webapp appBase

Tomcat sample configuration:

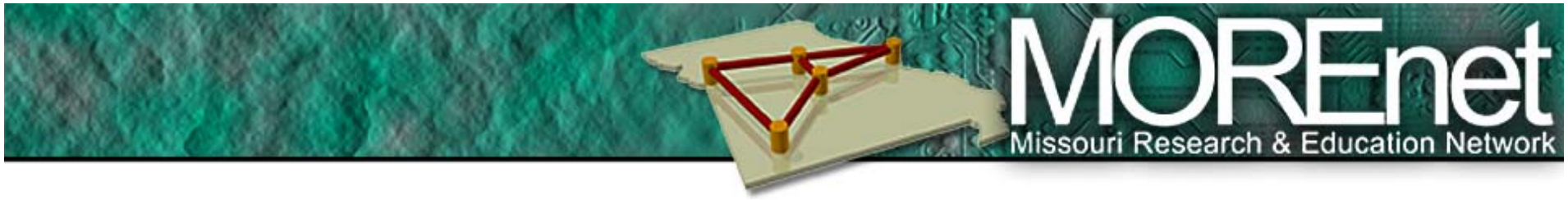
```
<Host name="localhost" debug="0"
   appBase="C:\WebPublish\localhost\www"
   WorkDir="C:\WebPublish\localhost\tomcat\work"
   unpackWARs="true" swallowOutput="true"
   autoDeploy="true" liveDeploy="false"
   deployXML="false">
```



Sharing the web site docBase and webapp appBase

Be careful! Don’t expose the web application source files through Apache.

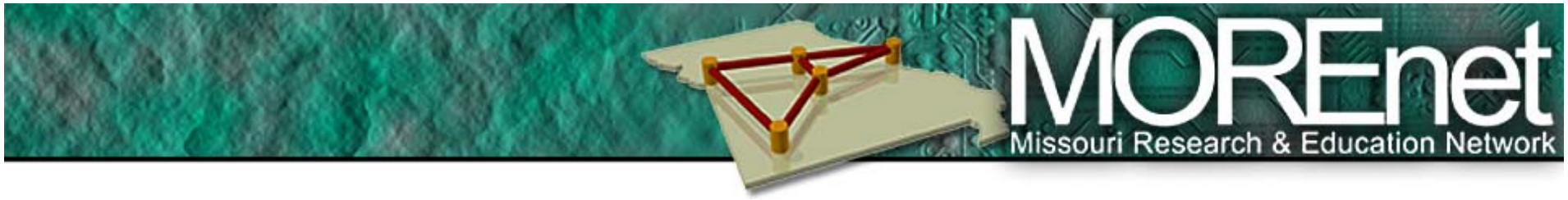
```
# Prevent Apache from serving web
# application source files
<LocationMatch "/WEB-INF/">
    AllowOverride None
    deny from all
</LocationMatch>
```



Configure Apache to Process SSI in Web Applications

With careful configuration, customers benefit from:

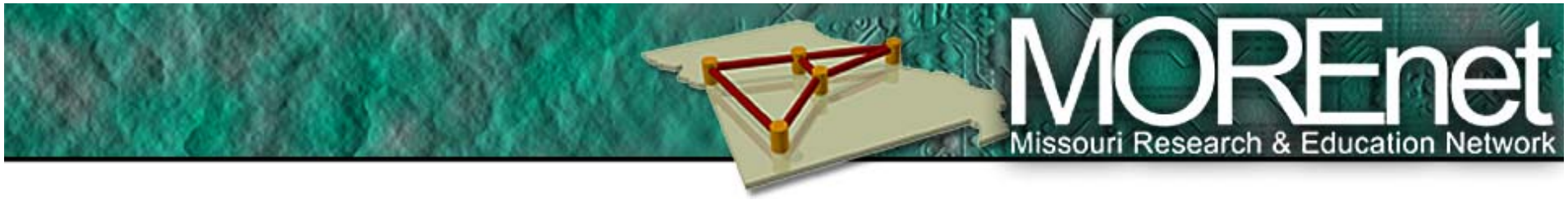
- Consistent design with use of SSI for common components such as headers, footers, and navigation.
- SSI works throughout the site, even in web applications.



Configure Apache to Process SSI in Web Applications

Apache sample configuration:

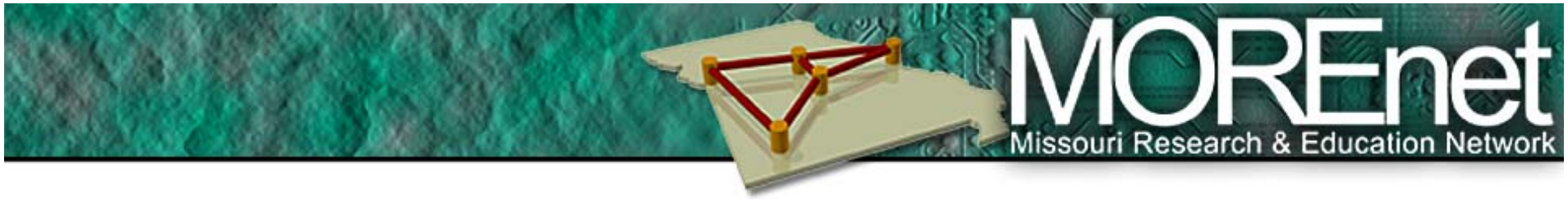
```
# Process SSI in struts/jsp files
<LocationMatch ".*\.(do|jsp)$">
  Options Includes
  AddOutputFilterByType INCLUDES text/html
</LocationMatch>
```



Configure Apache to Process SSI in Web Applications

Apache sample config:

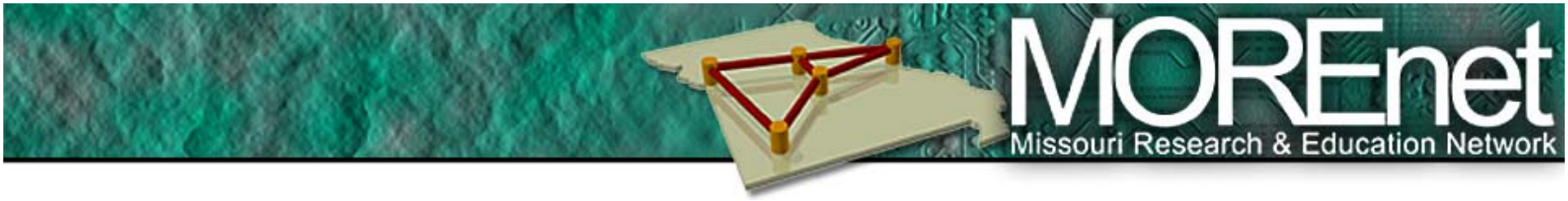
```
# Process SSI in servlets
<LocationMatch "^/[^\/]+/servlet/">
  Options Includes
  AddOutputFilterByType INCLUDES text/html
</LocationMatch>
```



Leverage mod_jk to Simplify Integration

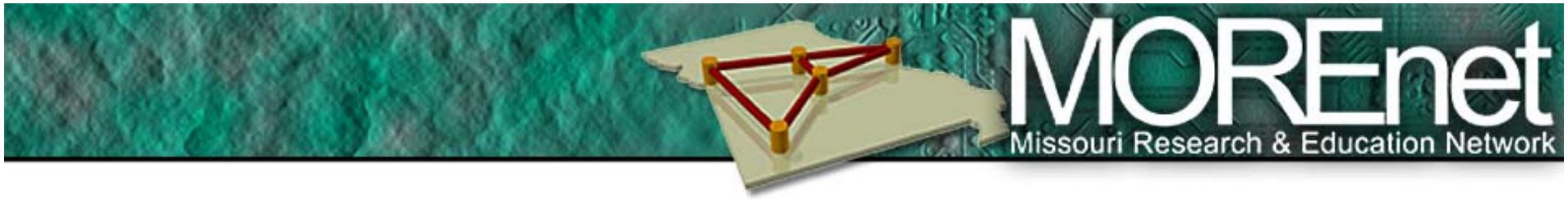
Mod_jk directives simplify web site and web application integration:

- JkAutoAlias directive automatically maps web application context directories into Apache's document space.
- JkAutoAlias also implements security measures. It automatically prevents access to web application `.war` files and `/WEB-INF` directories.



Leverage mod_jk to Simplify Integration

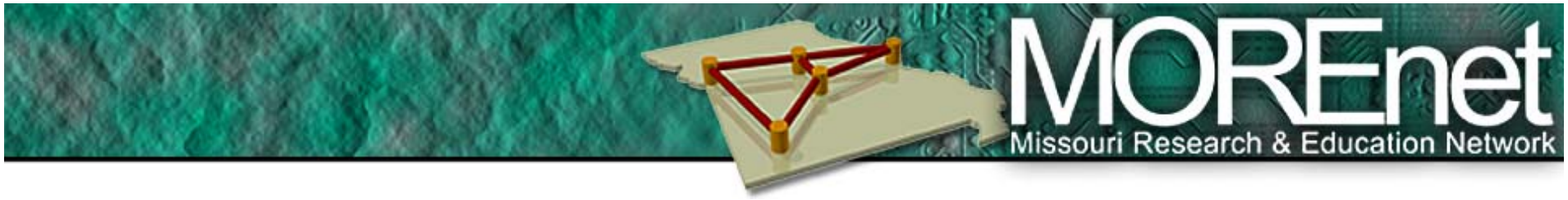
- JkMount allows you to automatically mount file types to the connector, thereby eliminating lots of manual editing of the virtual host for web applications.



Leverage mod_jk to Simplify Integration

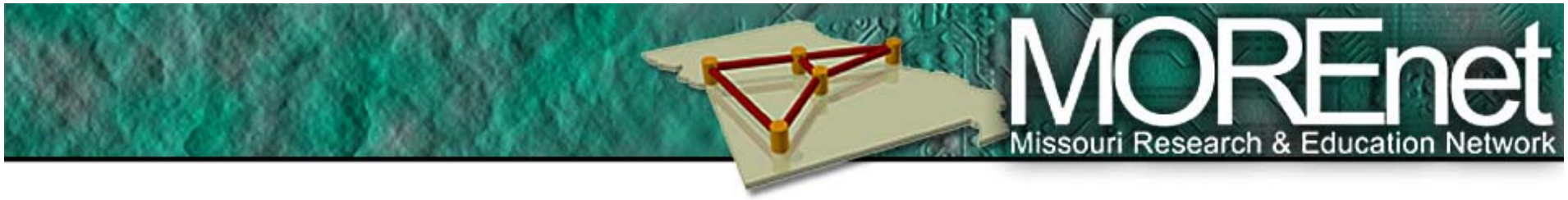
Apache sample config:

```
# Map webapp context directories into
# Apache's document space
JkAutoAlias "/WebPublish/localhost/www"
# mount .jsp, servlet, and struts files to the worker
JkMount /*.jsp tomcat
JkMount /*/servlet/ tomcat
JkMount /*.do tomcat
# mount the tomcat manager to the worker
JkMount /manager/* tomcat
```



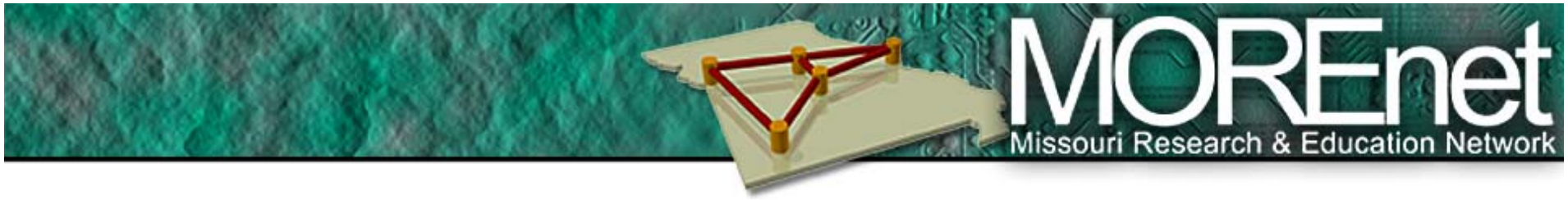
Providing Database and Mail Services

Web application access to databases and mail services should be secure and authenticated. By configuring MySQL and Tomcat properly, customer web applications have secure and simplified access to these resources.



Providing Database and Mail Services

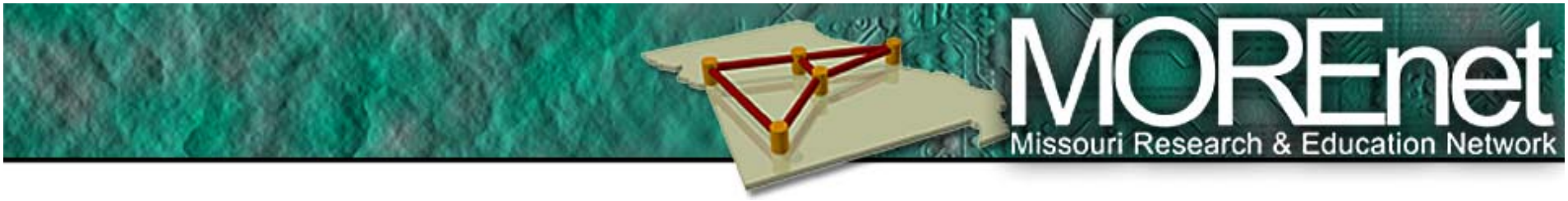
- Tomcat supports JNDI (Java Naming and Directory Interface) for access to database and mail services.
- JNDI abstracts these resources so that web applications can work without modification on the desktop and production server.
- Configured per <host> in Tomcat configuration.



Providing Database and Mail Services

Customers and administrators benefits:

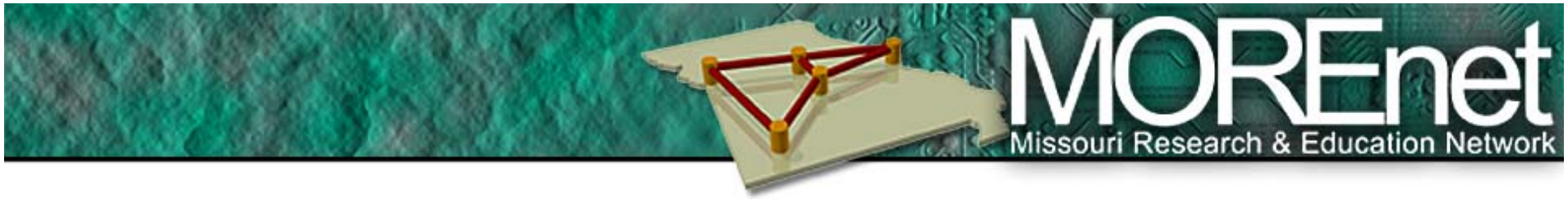
- Access is configured in a secure, central location.
- Usernames and passwords are abstracted instead of scattered throughout web applications.
- Web applications have improved performance since Tomcat implements database connection pooling.



Database Services

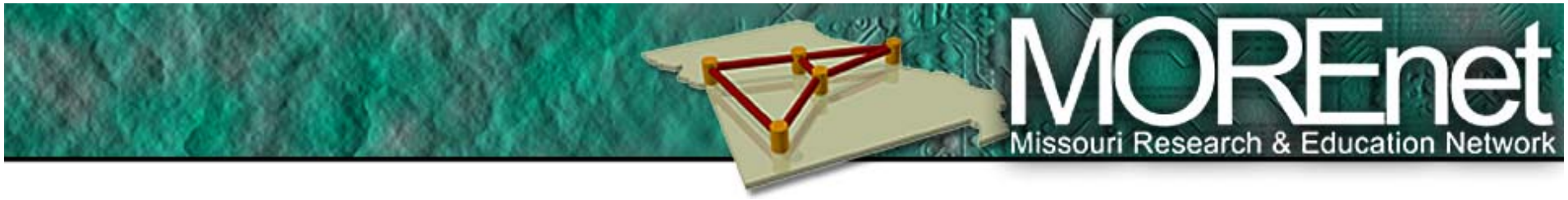
Sample Tomcat configuration:

```
<host …
  <DefaultContext …
    <Resource name="jdbc/data" auth="CONTAINER"
              type="javax.sql.DataSource"/>
      <ResourceParams name="jdbc/data">
        <parameter>
          <name>factory</name>
  <value>org.apache.commons.dbcp.BasicDataSourceFactory
  </value>
        </parameter>
```



Database Services

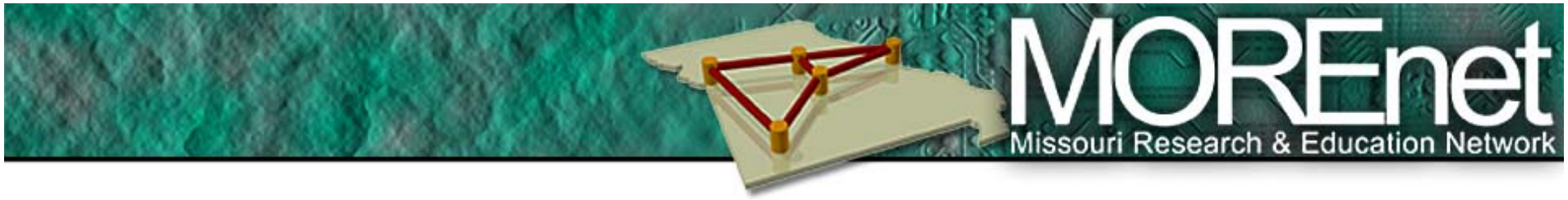
```
<parameter>
  <name>username</name>
  <value>xxxxxx</value>
</parameter>
<parameter>
  <name>password</name>
  <value>xxxxxx</value>
</parameter>
<parameter>
  <name>driverClassName</name>
  <value>com.mysql.jdbc.Driver</value>
</parameter>
```



Database Services

```
<parameter>
  <name>url</name>
  <value>
    jdbc:mysql://localhost/local?autoReconnect=true
  </value>
</parameter>
<parameter>
  <name>maxIdle</name><value>10</value>
</parameter>
<parameter>
  <name>maxActive</name><value>50</value>
</parameter>
```

… as documented in Tomcat default config



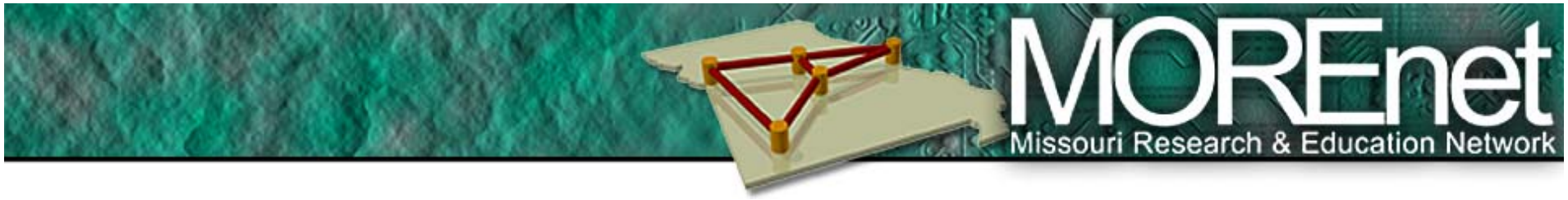
Mail Services

Sample Tomcat configuration:

```
<host …
  <DefaultContext …
    <Resource name="mail/send" auth="CONTAINER"
       type="javax.mail.internet.MimePartDataSource"/>
      <ResourceParams name="mail/send">
        <parameter>
          <name>factory</name>
          <value>
            org.apache.naming.factory.SendMailFactory
          </value>
        </parameter>
```

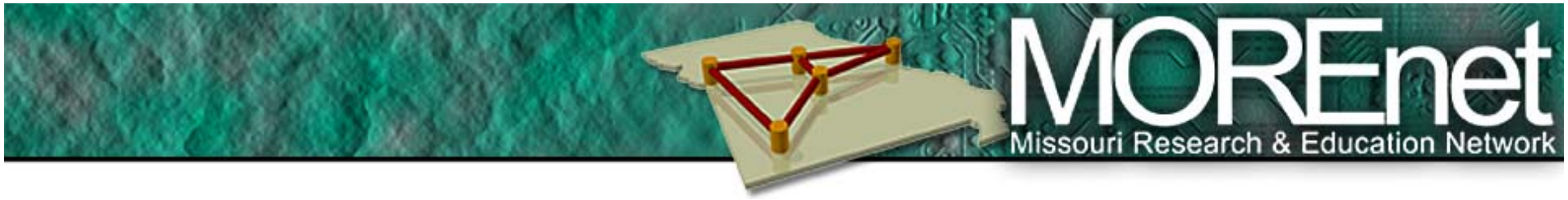
Mail Services

```
  <parameter>
    <name>mail.smtp.host</name>
    <value>host.domain</value>
  </parameter>
  <parameter>
    <name>mail.smtp.user</name>
    <value>userid</value>
  </parameter>
  <parameter>
    <name>mail.from</name>
    <value>userid@domain</value>
  </parameter>
</ResourceParams>
```



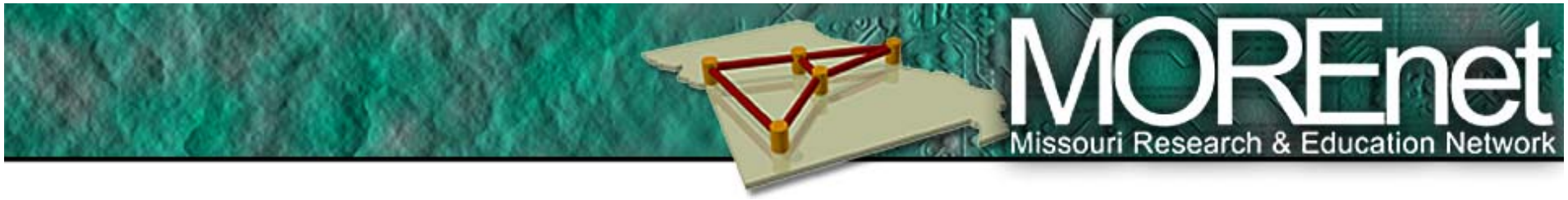
Securing MySQL With Grant Tables and Permissions

- On the host system, sysads ensure that the database is properly configured for secure access.
- Customers are provided the JNDI resource name for web applications and one MySQL account for remotely managing the database on the host.



Securing MySQL With Grant Tables and Permissions

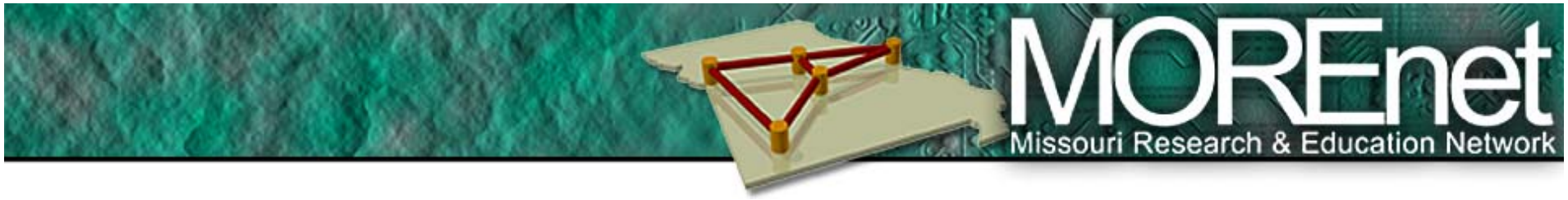
- On the desktop, customers have MySQL server administrative privileges.
- Default MySQL grants are wide-open.
- Customers are not sysads. Provide them with examples of secure grants.
- Advise restricted access to the localhost.



Securing MySQL With Grant Tables and Permissions

See Appendix D for an example grants table.

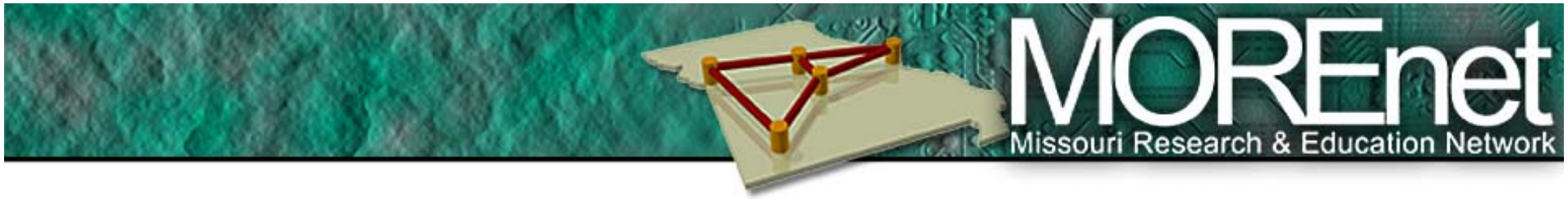
- Note restriction to localhost Host
- Only root user is given all privileges



Managing Tomcat

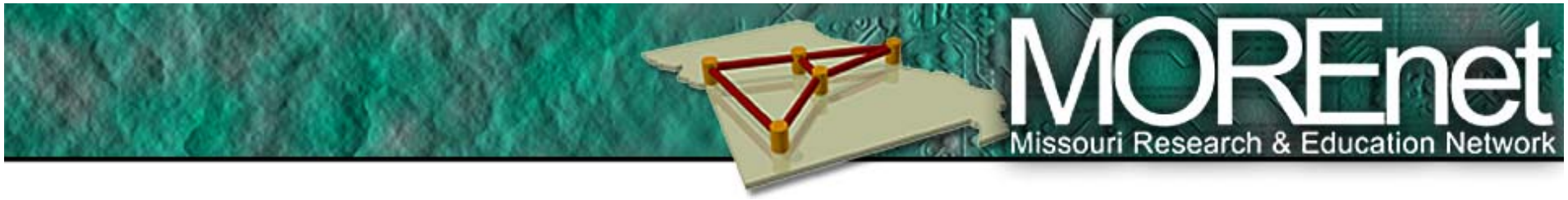
On the host system and desktop:

- Use a security policy for strict control to protect your system.
- Use the Tomcat Manager for web application management.



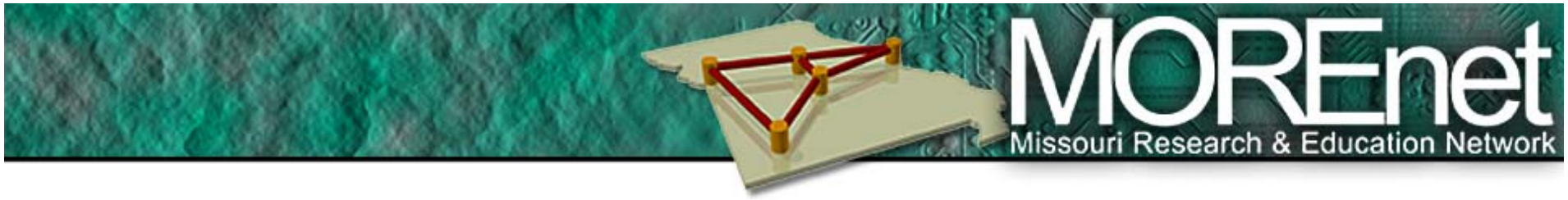
Enforce Security with the Java SecurityManager

Tomcat provides a mechanism for using the Java SecurityManager for fine-grained control over the operation of the servlet container. A production instance of Tomcat should run with a carefully constructed security policy. This policy can be easily modified for the desktop environment.



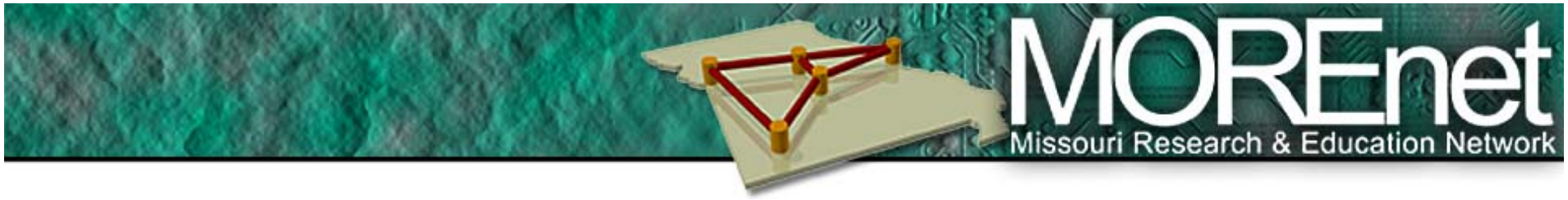
Enforce Security with the Java SecurityManager

- This is a complex topic! Review Glenn Nielsen's ApacheCon US 2002 presentation "Tomcat Server and Application Security".
- Tomcat distribution provides minimal security in its security policy.
- See Appendix C for a sample security policy for the desktop.



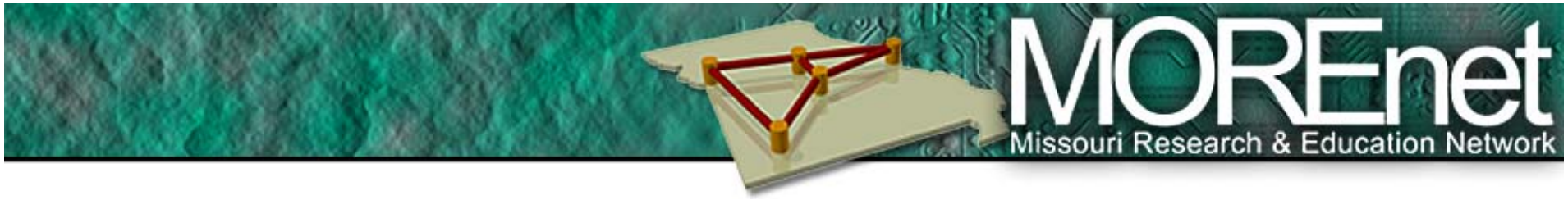
Delegate Webapp Support With the Tomcat Manager

- Customers need the ability to manage their web applications. Without a management tool, support personnel will be overloaded with requests for starting, stopping, removing, and deploying web applications.
- The Tomcat Manager provides all of these abilities in an authenticated web interface.



Delegate Webapp Support With the Tomcat Manager

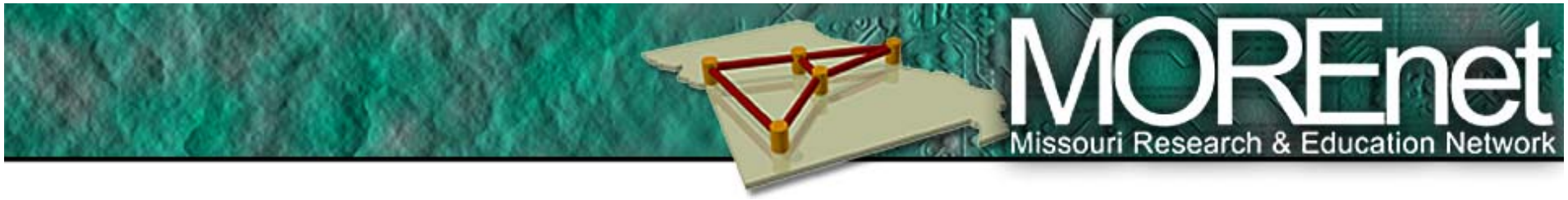
- Provide each customer with a username and password for managing applications.
- Configure Tomcat's manager to use MySQL for authenticating users and add the usernames and passwords to the database. See Appendix E for database schema.
- Provide customers with extra security on the desktop by limiting Tomcat Manager access to the localhost.



Delegate Webapp Support With the Tomcat Manager

Tomcat config for Manager Authentication:

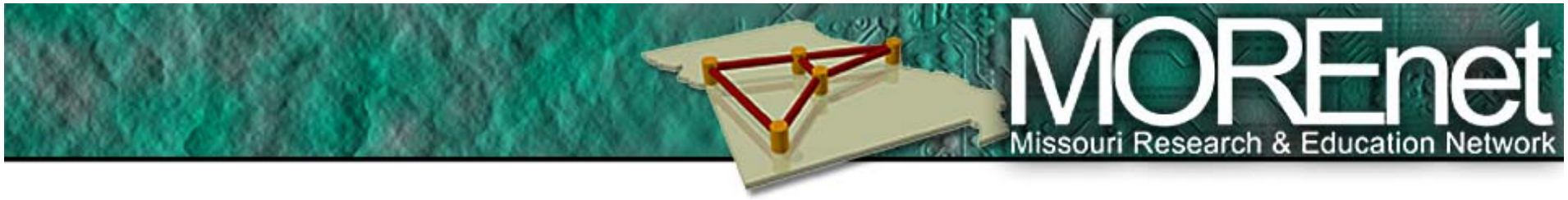
```
<host …
  <Realm className="org.apache.catalina.realm.JDBCRealm"
        debug="0" driverName="com.mysql.jdbc.Driver"
connectionURL="jdbc:mysql://localhost/local_realm?autoRe
  connect=true" digest="MD5"
        connectionName="username"
        connectionPassword="xxxxxx"
        userTable="users" userNameCol="user_name"
        userCredCol="user_pass"
        userRoleTable="user_roles"
        roleNameCol="role_name"/>
```



Delegate Webapp Support With the Tomcat Manager

Tomcat config for localhost-only access to Manager on the desktop:

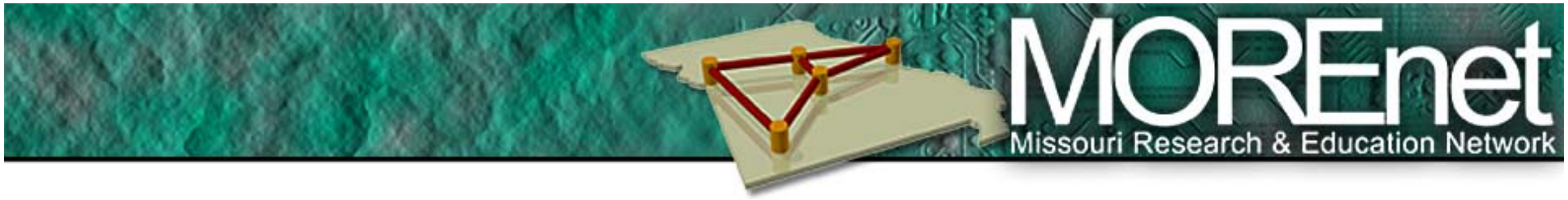
```
<host…
  <Context path="/manager"
    docBase="C:\Tomcat4.1\server\webapps\manager"
    debug="0" privileged="true">
    <!- only allow localhost access -->
    <Valve
 className="org.apache.catalina.valves.RemoteAddrValve"
 allow="127.0.0.1" />
        </Context>
```



Strategies for Helping Customers

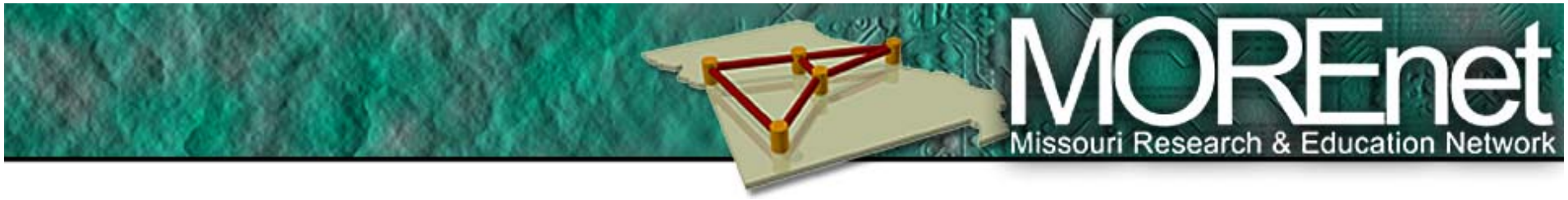
Consider the following:

- Apache, Tomcat and MySQL are NOT configured by default as described in this presentation.
- Customers may not have lots of technical experience.
- There are lots of details in the configurations there is little room for error.



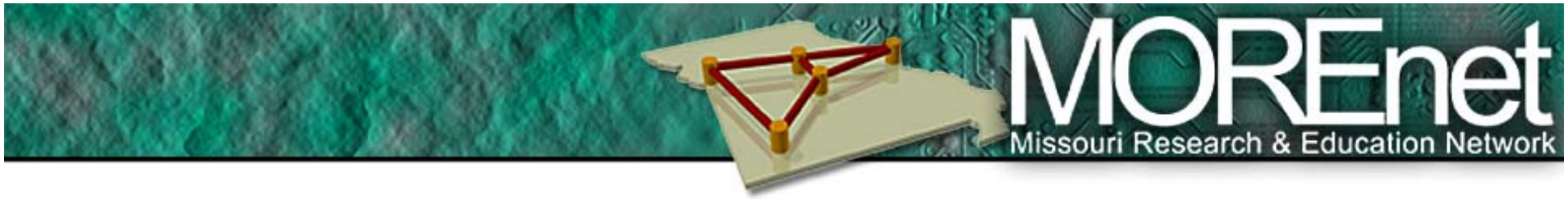
Strategies for Helping Customers

- Detailed instructions with example configuration files is the minimum you can do to help customers.
- Consider building installers that automate the installation and configuration process.



Strategies for Helping Customers

- Customers may experience difficulties if they want to use Tomcat embedded within an IDE since it may not be configurable as described in this presentation.
- If the IDE supports ANT, you may be able to overcome this by using targets that build and then deploy applications into a custom Tomcat instance outside the IDE.



Thanks for attending!

For last minute updates to presentation and handouts:

http://kinetic.more.net/web/javaserver/resources/wpapers/

For more information about this environment:

http://kinetic.more.net/

This presenation has ended.
Click anywhere to return.

_

